

Plano de Handoff Tecnico

Plataforma Juridico-Contabil Docker-First
8 Produtos | 52 Semanas | SLA 99.5%

Handoff completo para desenvolvimento de plataforma de IA aplicada ao direito e contabilidade brasileira. Arquitetura Docker Compose com caminho de expansao para Kubernetes, 8 produtos integrados sobre stack compartilhado, 14 fontes de dados publicos governamentais, e Decision Ledger com Merkle tree para auditoria regulatoria.

8

PRODUTOS

14

FONTES DE DADOS

52

SEMANAS

99.5%

SLA

Stack: Docker Compose + FastAPI + Neo4j + PostgreSQL

Frontend: Next.js 14 + TanStack Query

IA: RAG Multi-Agente + Modelo Bayesiano

Confidencial

Uso interno do time de desenvolvimento

Sumario Executivo



Este documento e o handoff tecnico completo para a construcao da **Plataforma Juridico-Contabil Docker-First**, uma suíte de 8 produtos de inteligencia artificial aplicada ao direito e a contabilidade brasileira. A arquitetura parte de Docker Compose como base operacional, com caminho de migracao definido para Kubernetes e servicos gerenciados em cloud quando o volume justificar a transicao. Todos os produtos compartilham uma infraestrutura comum de banco de dados, cache, busca vetorial, armazenamento de objetos, monitoramento e pipeline de dados, reduzindo o custo marginal de cada novo produto em aproximadamente 60%.

A plataforma combina tres capacidades fundamentais: (1) agentes de IA multi-agente com consenso ponderado para pesquisa jurisprudencial, (2) cruzamento automatico de demonstracoes financeiras com 14 fontes de dados publicos governamentais, e (3) monitoramento proativo com alertas multi-canal. Toda decisao automatizada gera registro auditavel no Decision Ledger com hash criptografico e Merkle tree, garantindo rastreabilidade completa para fins regulatorios (Bacen, CVM, LGPD).

Produtos da Plataforma

Matriz de produtos da plataforma

#	Produto	Categoria	Entrega Principal	SLA
1	LegalScore PJ	Rating de risco PJ	API + Dashboard + Decision Ledger	p95 < 1.5s
2	ContabillA	Auditoria contabil automatizada	Upload > relatorio PDF + API	p95 < 60s
3	ComplianceRadar	Monitoramento municipal	Dashboard geografico + alertas	99% alertas
4	TaxPredict	Previsao de desfecho tributario	API preditiva + PDF explicativo	p95 < 3s
5	LicitaWatch	Monitoramento de licitacoes	Dashboard + alertas + API	Diario
6	DanoBot	Calculo de danos socioeconômicos	Laudo PDF + API	p95 < 5s
7	PetiBot	Gerador de pecas processuais	Minuta + citações + dados	p95 < 10s
8	ConciliaIA	Analise de viabilidade de acordos	Faixas de valor + dados	p95 < 3s

Princípios Arquiteturais

1. **Compose today, K8s tomorrow:** cada serviço é um container isolado com health checks, resource limits e configuração externalizada. A transição exige apenas reescrever compose para Helm charts, sem mudar código.
2. **Dados reais, sem mock:** todos os pipelines ingerem de fontes públicas governamentais reais (DATAJUD, CAGED, SICONFI, PNCP, DATASUS, SNIS) com cadências definidas.
3. **Decision Ledger imutável:** toda decisão automatizada gera registro append-only com hash SHA-256, fontes consultadas e Merkle tree para verificação de integridade.
4. **LGPD by design:** pseudonimização no ingest, k-anonymity, right-to-erasure implementado, dados pessoais nunca em texto claro no grafo.
5. **Custo controlado:** cache estratégico de LLM, modelo local para classificação, embedding cache para RAG, rate limiting por tenant.

1. Visao Geral da Plataforma

A plataforma opera como um stack de microservicos containerizados comunicando-se via rede Docker interna. A arquitetura segue o padrao de **plataforma compartilhada**: uma unica infraestrutura de dados e IA serve todos os 8 produtos, e cada produto adiciona apenas seu container de API e (opcionalmente) seu frontend. Isso significa que o custo de infraestrutura e pago uma vez, e cada novo produto agrega apenas compute incremental.

1.1 Componentes do Stack Compartilhado

Stack compartilhado - todos os produtos

Componente	Tecnologia	Papel	Recurso
PostgreSQL 16	Relacional	Dados estruturados, Ledger, metadados	4 GB RAM, 2 vCPU
PgBouncer	Pool de conexoes	Evitar estouro de conexoes (max 100)	256 MB RAM
Neo4j 5 Enterprise	Grafo	Relacoes societarias, processos, tribunais	8 GB RAM, 4 vCPU
OpenSearch 2.12	Busca full-text	Acordaos, legislacao, busca semantica	4 GB RAM, 2 vCPU
Redis 7 + Sentinel	Cache + Broker	Cache, sessoes, Celery broker	2 GB RAM, 1 vCPU
MinIO	S3-compativel	Data Lake (bronze/silver/gold), documentos	Disco sob demanda
ChromaDB	Vetorial	Embeddings para RAG	2 GB RAM, 1 vCPU
Ollama	LLM local	Classificacao, NER, sentiment	8 GB RAM, 2 vCPU
Traefik v3	Reverse Proxy	Routing, TLS, rate limiting	256 MB RAM
Prometheus + Grafana	Monitoramento	Metricas, alertas, dashboards	1 GB RAM
Loki + Promtail	Logging	Logs estruturados JSON	512 MB RAM
Celery + Beat	Orquestracao	Agendamento e execucao de pipelines	4 GB RAM, 2 vCPU
Flower	Celery UI	Monitoramento de tasks	128 MB RAM

1.2 Estrutura de Diretórios

```
juridico-platform/
├── docker/
│   ├── compose/
│   │   ├── base.yml          # Infraestrutura compartilhada
│   │   ├── monitoring.yml    # Prometheus + Grafana + Loki
│   │   ├── ingest.yml       # Celery workers + Redis
│   │   ├── ollama.yml        # LLM local para classificacao
│   │   └── traefik.yml        # Reverse proxy + TLS
│   ├── products/
│   │   ├── legalscore/
│   │   │   ├── Dockerfile
│   │   │   └── compose.override.yml
│   │   ├── contabilialia/
│   │   ├── compliance-radar/
│   │   └── taxpredict/
│   └── config/
│       ├── neo4j.conf
│       ├── redis-sentinel.conf
│       ├── prometheus.yml
│       └── alertmanager.yml
├── services/
│   ├── gateway/              # API Gateway compartilhado (FastAPI)
│   ├── ai-engine/            # Motor de IA (RAG, consenso, cache)
│   ├── ingest/               # Pipeline de ingestao de dados
│   ├── scoring/              # Score Engine (LegalScore)
│   ├── audit/                # Cross-check engine (Contabilialia)
│   ├── compliance/           # Anomaly detection (ComplianceRadar)
│   ├── taxpredict/           # Bayesian model (TaxPredict)
│   ├── shared/               # Bibliotecas compartilhadas
│   │   ├── db/
│   │   ├── vector_store/
│   │   ├── consensus/
│   │   ├── ledger/
│   │   └── http_client/
├── frontend/
│   ├── apps/
│   │   ├── legalscore/      # Next.js
│   │   ├── contabilialia/
│   │   ├── compliance-radar/
│   │   └── taxpredict/
│   └── packages/            # Shared UI components
├── infra/
│   ├── terraform/           # IaC para VM provisioning
│   └── scripts/              # setup-docker.sh, backup.sh
├── data/
│   ├── bronze/              # MinIO: dados brutos
│   ├── silver/              # MinIO: limpos/validados
│   └── gold/                 # MinIO: agregados/prontos
├── tests/
│   ├── e2e/
│   ├── integration/
│   └── load/                 # Cenarios Locust
├── secrets/                  # NUNCA commitados
├── .env.example
├── Makefile
└── docker-compose.yml        # Compose principal
```

2. Infraestrutura Docker - Stack Base

Todos os refinamentos do analista ja aplicados: PgBouncer, Neo4j config via arquivo, Redis Sentinel, Docker Secrets, MinIO lifecycle.

2.1 PostgreSQL + PgBouncer

O PgBouncer e obrigatorio desde o dia 1. Com Celery workers (4+ conexoes persistentes) + API (uvicorn workers) + eventuais workers de ingest paralelo, o PostgreSQL rapidamente atinge o limite padrao de 100 conexoes. O PgBouncer gerencia um pool de conexoes eficientes, mantendo no maximo 20 conexoes reais ao PostgreSQL enquanto aceita centenas de clientes.

```
# docker/compose/base.yml

postgres:
  image: postgres:16-alpine
  container_name: postgres
  restart: unless-stopped
  environment:
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    POSTGRES_DB: ${POSTGRES_DB}
  volumes:
    - pg_data:/var/lib/postgresql/data
    - ./scripts/bootstrap-db.sql:/docker-entrypoint-initdb.d/01-init.sql
  ports:
    - "5432:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
    interval: 10s
    timeout: 5s
    retries: 5
  deploy:
    resources:
      limits:
        memory: 4G
        cpus: "2"

pgbouncer:
  image: edoburu/pgbouncer:1.20
  container_name: pgbouncer
  restart: unless-stopped
  environment:
    PGBOUNCER_DATABASE: ${POSTGRES_DB}
    PGBOUNCER_USER: ${POSTGRES_USER}
    PGBOUNCER_PASSWORD: ${POSTGRES_PASSWORD}
    PGBOUNCER_SERVERS: postgres:5432
    PGBOUNCER_AUTH_TYPE: md5
    PGBOUNCER_POOL_MODE: transaction
    PGBOUNCER_MAX_CLIENT_CONN: 1000
    PGBOUNCER_DEFAULT_POOL_SIZE: 20
  ports:
    - "6432:6432"
  depends_on:
    postgres:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -h localhost -p 6432"]
    interval: 10s
    timeout: 5s
    retries: 5
```

[i] Conexão via PgBouncer

Todos os serviços devem usar `postgresql://...@pgbouncer:6432/...` em vez de conectar diretamente ao PostgreSQL na porta 5432. A porta 5432 fica exposta apenas para administração e backups diretos.

2.2 Neo4j Enterprise com Config via Arquivo

O Neo4j 5 Enterprise (licença developer, gratuita até 1GB de grafo) permite clustering futuro. A configuração de memória passa por arquivo `neo4j.conf` montado como volume, evitando problemas com variáveis de ambiente e `snake_case`. O plugin APOC é carregado para procedimentos adicionais de graph analytics.

```
# docker/compose/base.yml
```

```
neo4j:
  image: neo4j:5-enterprise
  container_name: neo4j
  restart: unless-stopped
  volumes:
    - neo4j_data:/data
    - neo4j_logs:/logs
    - ./config/neo4j.conf:/etc/neo4j/neo4j.conf
  environment:
    NEO4J_AUTH: neo4j/${NEO4J_PASSWORD}
    NEO4J_server_mode: "SINGLE" # CORE quando cluster
  ports:
    - "7474:7474"
    - "7687:7687"
  healthcheck:
    test: ["CMD-SHELL", "cypher-shell -u neo4j -p ${NEO4J_PASSWORD} 'RETURN 1'"]
    interval: 15s
    timeout: 10s
    retries: 5
  deploy:
    resources:
      limits:
        memory: 8G
        cpus: "4"
```

```
# docker/config/neo4j.conf
server.memory.heap.initial_size=2G
server.memory.heap.max_size=4G
server.memory.pagecache.size=2G
dbms.tx_log.rotation.retention_policy=100M size, 7 days
dbms.security.procedures.unrestricted=apoc.*
dbms.connector.bolt.advertised_address=neo4j:7687
dbms.connector.http.advertised_address=neo4j:7474
```

2.3 Redis + Sentinel

Redis com Sentinel desde o início cria o padrão de alta disponibilidade que será migrado para cluster posteriormente. A senha é gerenciada via Docker Secrets em produção, com fallback para `.env` em desenvolvimento. O Sentinel monitora o Redis master e executa failover automático se necessário.

```
# docker/compose/base.yml

redis:
  image: redis:7-alpine
  container_name: redis
  restart: unless-stopped
  command: >
    redis-server
    --maxmemory 2gb
    --maxmemory-policy allkeys-lru
    --appendonly yes
    --requirepass ${REDIS_PASSWORD}
  volumes:
    - redis_data:/data
  ports:
    - "6379:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
  deploy:
    resources:
      limits:
        memory: 2G
        cpus: "1"

# docker/compose/redis-sentinel.yml
sentinel:
  image: redis:7-alpine
  container_name: redis-sentinel
  restart: unless-stopped
  command: redis-server /etc/redis/sentinel.conf --sentinel
  volumes:
    - ./config/redis-sentinel.conf:/etc/redis/sentinel.conf
  ports:
    - "26379:26379"
  depends_on:
    redis:
      condition: service_healthy
```

```
# docker/config/redis-sentinel.conf
sentinel monitor mymaster redis 6379 1
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 60000
sentinel parallel-syncs mymaster 1
requirepass ${REDIS_PASSWORD}
```

2.4 MinIO com Versioning e Lifecycle

O MinIO funciona como S3-compatível para o Data Lake em três camadas (bronze, silver, gold). O versioning permite rastrear mudanças em dados ingeridos, e a lifecycle policy evita que o disco encha com dados antigos. O script de inicialização cria os buckets e configura as políticas automaticamente.

```
# docker/compose/base.yml

minio:
  image: minio/minio:latest
  container_name: minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_USER}
    MINIO_ROOT_PASSWORD: ${MINIO_PASSWORD}
  volumes:
    - minio_data:/data
  ports:
    - "9000:9000"
    - "9001:9001"
  healthcheck:
    test: ["CMD", "mc", "ready", "local"]
    interval: 15s
    timeout: 10s
    retries: 5

minio-init:
  image: minio/mc:latest
  depends_on:
    minio:
      condition: service_healthy
  entrypoint: >
    /bin/sh -c "
    mc alias set local http://minio:9000 ${MINIO_USER} ${MINIO_PASSWORD};
    mc mb local/bronze --ignore-existing;
    mc mb local/silver --ignore-existing;
    mc mb local/gold --ignore-existing;
    mc mb local/documents --ignore-existing;
    mc version enable local/bronze;
    mc version enable local/silver;
    mc version enable local/gold;
    mc anonymous set download local/bronze;
    exit 0;
    "
```

2.5 Traefik v3 com TLS

O Traefik atua como reverse proxy e roteador baseado em labels dos containers. Em produção, o Let's Encrypt emite certificados TLS automaticamente via ACME TLS challenge. O arquivo `acme.json` deve ter permissões 600 e ser persistido em volume, caso contrário o Let's Encrypt revoga o certificado a cada reinício da VM.

```
# docker/compose/traefik.yml
```

```
traefik:
  image: traefik:v3.0
  container_name: traefik
  restart: unless-stopped
  command:
    - "--api.insecure=true"
    - "--providers.docker=true"
    - "--providers.docker.exposedbydefault=false"
    - "--entrypoints.web.address=:80"
    - "--entrypoints.websecure.address=:443"
    - "--certificatesresolvers.letsencrypt.acme.tlschallenge=true"
    - "--certificatesresolvers.letsencrypt.acme.email=${LETSENCRYPT_EMAIL}"
    - "--certificatesresolvers.letsencrypt.acme.storage=/data/acme/acme.json"
  ports:
    - "80:80"
    - "443:443"
    - "8080:8080"
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - traefik_acme:/data/acme
  deploy:
    resources:
      limits:
        memory: 256M
        cpus: "0.5"
```

[!] Permissões do acme.json

Execute `chmod 600 data/acme/acme.json` antes de iniciar o Traefik. Sem isso, o Let's Encrypt recusa emissor de certificados e o TLS falha em produção.

3. Pipeline de Dados

Orquestracao via Celery Beat com gatilhos definidos para migracao para Airflow.

3.1 Celery Beat Schedule

O Celery Beat funciona como orquestrador inicial, substituindo o Airflow na Fase 0-2. Cada tarefa de ingestao e um Celery task com retry exponencial, timeout configurado e logging estruturado. O gatilho para migracao para Airflow e explicito: mais de 15 DAGs com dependencias cruzadas, ou mais de 5M registros por DAG.

```

# services/ingest/schedule.py
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    # === DIARIO (02:00 BRT) ===
    "datajud-daily-ingest": {
        "task": "ingest.tasks.datajud.run_daily_ingest",
        "schedule": crontab(hour=2, minute=0),
        "options": {"queue": "daily"},
    },
    "pncp-daily-ingest": {
        "task": "ingest.tasks.pncp.run_daily_ingest",
        "schedule": crontab(hour=2, minute=30),
        "options": {"queue": "daily"},
    },
    "cnpj-daily-sync": {
        "task": "ingest.tasks.cnpj.run_daily_sync",
        "schedule": crontab(hour=3, minute=0),
        "options": {"queue": "daily"},
    },
    # === MENSAL ===
    "siconfi-monthly-ingest": {
        "task": "ingest.tasks.siconfi.run_monthly_ingest",
        "schedule": crontab(hour=3, minute=0, day_of_month="5"),
        "options": {"queue": "monthly"},
    },
    "caged-monthly-ingest": {
        "task": "ingest.tasks.caged.run_monthly_ingest",
        "schedule": crontab(hour=3, minute=30, day_of_month="10"),
        "options": {"queue": "monthly"},
    },
    # === SEMANAL ===
    "pgfn-weekly-ingest": {
        "task": "ingest.tasks.pgfn.run_weekly_ingest",
        "schedule": crontab(hour=4, minute=0, day_of_week="1"),
        "options": {"queue": "weekly"},
    },
    # === HORARIO ===
    "transparencia-hourly-check": {
        "task": "ingest.tasks.transparencia.run_hourly_check",
        "schedule": crontab(minute=0),
        "options": {"queue": "hourly"},
    },
    # === POS-INGEST ===
    "compliance-anomaly-scan": {
        "task": "compliance.tasks.run_anomaly_scan",
        "schedule": crontab(hour=5, minute=0),
        "options": {"queue": "daily"},
    },
    "scoring-batch-update": {
        "task": "scoring.tasks.run_batch_recalc",
        "schedule": crontab(hour=6, minute=0, day_of_week="0,3"),
        "options": {"queue": "batch"},
    },
}

```

3.2 Fontes de Dados e Cadencias

Fontes de dados publicos com cadencias e destinos

Fonte	Dominio	Cadencia	Destino	Lag Típico
Receita Federal	CNPJ/Cadastro	Diaria	PostgreSQL + Neo4j	1 dia
DATAJUD	Processos/Jurisprudencia	Diaria	OpenSearch + ChromaDB	2 dias
PGFN	Divida ativa	Semanal	PostgreSQL	7 dias
CAGED (MTE)	Emprego	Mensal	PostgreSQL + Neo4j	40 dias
SICONFI (STN)	Execucao orcamentaria	Mensal	PostgreSQL + MinIO	365 dias
PNCP	Licitacoes	Diaria	PostgreSQL + Neo4j	30 dias
DATASUS/SIH	Saude	Mensal	PostgreSQL	90 dias
SNIS (MDR)	Saneamento	Anual	PostgreSQL	548 dias
INEP	Educacao	Anual	PostgreSQL	365 dias
ComexStat	Comercio exterior	Mensal	PostgreSQL	30 dias
Portal Transparencia	Receitas/Despesas	Horaria	Redis cache	1 dia
Câmara Deputados	Legislacao	On-demand	ChromaDB	Tempo real

3.3 Resiliencia: DATAJUD Fallback

O DATAJUD é notoriamente instável, com quedas frequentes que podem durar horas. O pipeline implementa retry exponencial com three-tier fallback: (1) tentativa primária na API oficial, (2) cache Redis dos últimos 2 dias, (3) retry agendado para o dia seguinte. O score parcial é gerado com dados cacheados e recebe flag "dados desatualizados" para que o consumidor saiba o nível de confiança.

```
# services/ingest/tasks/datajud.py
import requests
from tenacity import retry, stop_after_attempt, wait_exponential
from celery import group
from datetime import datetime, timedelta

@app.task(
    bind=True,
    autoretry_for=(requests.RequestException,),
    retry_kwargs={'max_attempts': 5}
)
def run_daily_ingest(self, date: str = None):
    if not date:
        date = (datetime.utcnow() - timedelta(days=1)).strftime('%Y-%m-%d')

    try:
        response = requests.get(
            f"{settings.DATAJUD_API_URL}/processos",
            params={"data_julgamento": date},
            timeout=30,
            headers={"Authorization": f"Bearer {settings.DATAJUD_TOKEN}"},
        )
        response.raise_for_status()
        data = response.json()

    except (requests.RequestException, ValueError):
        # Fallback 1: cache Redis (últimos 2 dias)
        cached = get_redis().get(f"datajud:{date}")
        if cached:
            logger.warning(f"Fallback para cache Redis de {date}")
            data = json.loads(cached)
        else:
            # Fallback 2: retry com escalonamento
            self.retry(
                exc=DatajudUnreachable("API fora de serviço"),
                countdown=60 * 60 # Retry em 1h
            )

    # Processamento normal com pseudonimização
    for record in data.get('items', []):
        safe = pseudonymize_process_record(record)
        process_single_record.delay(safe)
```

[!] Gatilho de Migração para Airflow

Migrar de Celery Beat para Airflow quando qualquer uma destas condições for atingida: (1) mais de 15 DAGs com dependências cruzadas, (2) mais de 5M registros processados por DAG, (3) mais de 3 fontes com SLA menor que 95% que precisam de retries nativos e SLA hooks.

3.4 MinIO Lifecycle Policy

Para evitar que o disco encha, configure lifecycle policies no MinIO. Dados na camada bronze (brutos) são retidos por 90 dias e depois excluídos automaticamente. Dados na camada silver e gold são retidos por 365 dias. O script de setup aplica essas políticas na inicialização.

```
# scripts/setup-minio.sh
set -e
mc alias set local http://minio:9000 $MINIO_USER $MINIO_PASSWORD

# Lifecycle: bronze = 90 dias, silver/gold = 365 dias
cat > /tmp/lifecycle-bronze.json << 'EOF'
{
  "Rules": [{
    "ID": "delete-bronze-90d",
    "Status": "Enabled",
    "Expiration": {"Days": 90},
    "Filter": {}
  }]
}
EOF

cat > /tmp/lifecycle-silver.json << 'EOF'
{
  "Rules": [{
    "ID": "delete-silver-365d",
    "Status": "Enabled",
    "Expiration": {"Days": 365},
    "Filter": {}
  }]
}
EOF

mc ilm import local/bronze /tmp/lifecycle-bronze.json
mc ilm import local/silver /tmp/lifecycle-silver.json
mc ilm import local/gold /tmp/lifecycle-silver.json
echo "Lifecycle policies aplicadas com sucesso"
```

3.5 Pseudonimização LGPD no Pipeline

Todo registro ingerido passa por pseudonimização antes de ser armazenado em qualquer banco de dados. CPFs e nomes de pessoas são substituídos por hashes SHA-256 truncados. Endereços e CEPs são removidos completamente. CNPJs são mantidos em texto claro por serem dados públicos por natureza (disponíveis na Receita Federal). No Neo4j, apenas os dados pseudonimizados entram no grafo, garantindo que uma eventual vulnerabilidade de acesso ao banco nunca exponha dados pessoais.

```

# services/ingest/processors/datajud.py
import hashlib
from typing import Dict, Any

def pseudonymize_process_record(record: Dict[str, Any]) → Dict[str, Any]:
    safe_record = record.copy()

    # CPF → hash (12 caracteres)
    if 'parte_cpf' in safe_record:
        safe_record['parte_cpf_hash'] = hashlib.sha256(
            safe_record.pop('parte_cpf').encode()
        ).hexdigest()[:12]

    # CNPJ → mantido (dados publicos)
    # Nome da parte → hash
    if 'parte_nome' in safe_record:
        safe_record['parte_nome_hash'] = hashlib.sha256(
            safe_record.pop('parte_nome').encode()
        ).hexdigest()[:10]

    # Endereco e CEP → removidos
    safe_record.pop('endereco', None)
    safe_record.pop('cep', None)

    return safe_record

```

4. Motor de IA Compartilhado

Otimizações de custo e performance para LLM, embeddings e roteamento de modelos.

4.1 LLM Cache Estrategico

Cada chamada ao LLM custa dinheiro. O cache não deve ser apenas por prompt literal, mas por hash do input + modelo + versão do prompt template. Isso garante que: (1) prompts idênticos não gerem duplicatas, (2) mudanças no template invalidam o cache automaticamente, (3) o TTL de 30 dias equilibra economia com frescor dos dados. O cache Redis com chave SHA-256 e TTL configurável é a implementação recomendada.

```
# services/shared/ai/cache.py
import hashlib
import json
import redis
import logging
from functools import wraps

logger = logging.getLogger(__name__)

class LLMemoizer:
    def __init__(self, redis_url: str):
        self.r = redis.from_url(redis_url)

    def __call__(self, model: str = "gpt-4"):
        def decorator(func):
            @wraps(func)
            def wrapper(*args, **kwargs):
                prompt = kwargs.get('prompt') or args[0]
                template_ver = getattr(func, '__version__', '1')
                cache_key = hashlib.sha256(
                    f"{model}:{template_ver}:{str(prompt)}".encode()
                ).hexdigest()

                cached = self.r.get(f"llm:{cache_key}")
                if cached:
                    logger.info(f"LLM cache HIT: {cache_key[:8]}...")
                    return json.loads(cached)

                result = func(*args, **kwargs)
                self.r.setex(
                    f"llm:{cache_key}",
                    30 * 24 * 3600, # 30 dias
                    json.dumps(result)
                )
                return result
            return wrapper
        return decorator

# Uso:
memoizer = LLMemoizer(os.environ['REDIS_URL'])

@memoizer(model="gpt-4")
@__version__ = "1"
def generate_legal_opinion(prompt: str, **kwargs):
    # chamada ao LLM
    pass
```

4.2 Router de Modelos: Local vs API

A maior economia de custo vem do roteamento inteligente de tarefas. Tasks de classificacao (favoravel/desfavoravel, categoria de processo, NER) usam modelo open-source local via Ollama (Llama 3 8B), com custo zero. Tasks de geracao, raciocinio e elaboracao de textos juridicos usam API paga (GPT-4o, Claude). O LLMRouter decide automaticamente com base no tipo de tarefa.

Roteamento de modelos por tipo de tarefa

Tipo de Tarefa	Roteamento	Modelo	Custo Estimado
Classificacao (sentimento, categoria)	Local	Llama 3 8B via Ollama	Zero
NER (entidades em acordaos)	Local	Llama 3 8B via Ollama	Zero
Embeddings (RAG)	Local	BGE-M3 via Ollama	Zero
Geracao de texto juridico	API	GPT-4o / Claude	~\$0.01/req
Raciocinio complexo (consenso)	API	GPT-4o	~\$0.03/req
Sumarizacao de acordaos	API	GPT-4o-mini	~\$0.002/req

4.3 Embedding Cache com ChromaDB

O RAG e responsavel por 80% das chamadas de embedding. Re-embedar o mesmo documento 100 vezes e pura desperdicio. O ChromaDB com persistencia em PostgreSQL garante que embeddings nao se perdem entre reinicios, e a deduplicacao por hash do conteudo evita re-embedar documentos ja existentes. O collection usa cosine similarity como metrica de distancia, otimizada para busca semantica em textos juridicos.

```
# docker/compose/base.yml

chromadb:
  image: chromadb/chroma:latest
  container_name: chromadb
  restart: unless-stopped
  volumes:
    - chromadb_data:/chroma/chroma
  environment:
    - CHROMA_DB_IMPL=postgres
    - POSTGRES_HOST=pgbouncer
    - POSTGRES_PORT=6432
    - POSTGRES_USER=${POSTGRES_USER}
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - POSTGRES_DATABASE=chroma
  ports:
    - "8001:8000"
  healthcheck:
    test: ["CMD-SHELL", "curl -sf http://localhost:8000/api/v1/heartbeat"]
    interval: 10s
    timeout: 5s
    retries: 5
  depends_on:
    pgbouncer:
      condition: service_healthy
  deploy:
    resources:
      limits:
        memory: 2G
        cpus: "1"
```


5. Seguranca e Compliance

5.1 Docker Secrets vs .env

Em producao, segredos sao gerenciados via Docker Secrets (arquivos em `/run/secrets/`), nunca via variaveis de ambiente no compose file. Em desenvolvimento, `.env` com `direnv` e aceitavel. O helper `load_secret()` tenta Docker Secrets primeiro e fallback para variavel de ambiente, garantindo compatibilidade entre dev e prod.

```
# services/shared/config.py
from pathlib import Path

def load_secret(name: str) → str:
    path = Path("/run/secrets") / name
    if path.exists():
        return path.read_text().strip()
    import os
    return os.getenv(name.upper(), "")

# Uso em qualquer servico:
DB_PASSWORD = load_secret("db_password")
JWT_SECRET = load_secret("jwt_secret")
LLM_API_KEY = load_secret("llm_api_key")
```

5.2 Decision Ledger com Merkle Tree

O Decision Ledger e o diferencial competitivo da plataforma. Cada score gerado, cada analise de risco, cada predicacao cria um registro imutavel e auditavel. A Merkle tree permite verificar a integridade de qualquer entrada individual sem precisar re-hashar todo o ledger. Isso e essencial para: (1) auditorias regulatorias (Bacen, CVM), (2) disputas com clientes sobre resultados, (3) compliance com LGPD (rastreabilidade de decisoes automatizadas).

```

# services/shared/ledger/merkle.py
import hashlib
import json
from datetime import datetime
from typing import List, Dict, Any

class MerkleNode:
    def __init__(self, left=None, right=None, data=None):
        self.left = left
        self.right = right
        self.data = data
        self.hash = self._compute_hash()

    def _compute_hash(self) → str:
        if self.data:
            return hashlib.sha256(
                self.data.encode()).hexdigest()
        if self.left and self.right:
            combined = self.left.hash + self.right.hash
            return hashlib.sha256(combined.encode()).hexdigest()
        if self.left:
            return hashlib.sha256(
                (self.left.hash * 2).encode()).hexdigest()
        return hashlib.sha256(b"").hexdigest()

class DecisionLedger:
    def __init__(self, db_session):
        self.db = db_session
        self.entries: List[Dict] = []
        self.merkle_root: str = None

    def add_entry(self, request_id: str, user: str,
                  inputs: dict, outputs: dict,
                  sources: list, weights: dict):
        entry = {
            "request_id": request_id,
            "timestamp": datetime.utcnow().isoformat(),
            "user": user,
            "inputs_hash": hashlib.sha256(
                json.dumps(inputs, sort_keys=True).encode()
            ).hexdigest(),
            "outputs_hash": hashlib.sha256(
                json.dumps(outputs, sort_keys=True).encode()
            ).hexdigest(),
            "sources": sources,
            "weights_applied": weights,
            "merkle_proof": None, # preencho apos build
        }
        self.entries.append(entry)
        self._update_merkle()
        # Persistir no PostgreSQL (append-only)
        self._persist(entry)

    def _update_merkle(self):
        leaves = [
            hashlib.sha256(
                json.dumps(e, sort_keys=True).encode()
            ).hexdigest()
            for e in self.entries[-100:]
        ]
        self.merkle_root = self._build_tree(leaves)

    def verify_integrity(self, request_id: str) → bool:
        'Verifica se um registro nao foi alterado.'
        # Busca entry + Merkle proof do banco
        # Recalcula hash e compara com proof
        pass

```

5.3 LGPD: Checklist de Implementacao

Checklist LGPD com implementacoes e fases obrigatorias

Requisito LGPD	Implementacao	Status
Pseudonimizacao de dados pessoais	Hash SHA-256 no ingest, dados em claro nunca persistidos	Obrigatorio Fase 0
K-anonymity para dados agregados	Supressao de celulas com < 5 entidades (DadoSabedoria pattern)	Obrigatorio Fase 0
Right-to-erasure	Soft delete (flag) + hard delete apos 30 dias em PostgreSQL e Neo4j	Obrigatorio Fase 1
Consentimento por tenant	Tabela de consentimento com timestamp, IP e finalidade	Obrigatorio Fase 1
Audit trail de acessos	Log estruturado de toda query que acessa dados pessoais	Obrigatorio Fase 1
Criptografia em transito	TLS 1.3 em todas as conexoes (Traefik + Let's Encrypt)	Obrigatorio Fase 0
Criptografia em repouso	PostgreSQL: pgcrypto; MinIO: encryption at rest; Neo4j: encrypted db	Recomendado Fase 1
Retencao de dados (TTL)	Decision Ledger: 7 anos; Dados pessoais: 30 dias apos delecao	Obrigatorio Fase 0

6. Produto 1: LegalScore PJ v1.0

Sistema de rating de risco juridico-financeiro de Pessoas Juridicas. API REST + Dashboard Web. SLA 99.5%, latencia p95 menor que 1.5s.

6.1 Arquitetura

O LegalScore recebe um CNPJ, consulta multiplos fontes em paralelo (DATAJUD para processos, CAGED para emprego, PGFN para divida ativa, SICONFI para execucao municipal), cruza com dados do grafo Neo4j (relacoes societarias, historico de CNAE), aplica o modelo de scoring MLR calibrado por setor (CNAE 2 digitos), e gera um score de 0 a 1000 com intervalo de confianca. Toda operacao gera registro no Decision Ledger.

Componentes do LegalScore PJ

Componente	Tecnologia	Funcao
Score Engine	MLR custom (scikit-learn)	Calculo do score por CNAE com 7+ variaveis
Data Enrichment	Neo4j + OpenSearch + APIs	Consulta paralela de 5+ fontes
Decision Ledger	PostgreSQL append-only + Merkle	Trilha de auditoria imutavel
API	FastAPI + Uvicorn	6 endpoints REST documentados
Dashboard	Next.js 14 + TanStack Query	Scorecard, timeline, mapa de calor setorial
Notification	Webhook + Email + Slack	Alertas de mudanca de score

6.2 Score Engine (MLR por CNAE)

O modelo de scoring usa Regressao Linear Multipla (MLR) calibrada por setor CNAE de 2 digitos. Cada setor tem seu proprio conjunto de coeficientes, refletindo que os fatores de risco variam drasticamente entre, por exemplo, construcao civil e servicos de tecnologia. A calibracao inicial usa o metodo Delphi com especialistas juridicos, e e refinada iterativamente com dados de falencia e recuperacao judicial publicos.

```
# services/scoring/engine/model.py

class LegalScoreModel:
    def __init__(self, cnae_2digit: str):
        self.cnae = cnae_2digit
        self.coefficients = self._load_coefficients(cnae_2digit)

    def predict(self, features: dict) → dict:
        score = self.coefficients['intercept']
        feature_names = [
            'processos_ativos',
            'processos_trabalhistas',
            'divida_ativa_valor_log',
            'divida_ativa_crescimento',
            'saldo_emprego_12m',
            'capital_social_log',
            'processos_repetitivos'
        ]
        for name in feature_names:
            score += self.coefficients.get(name, 0) * features.get(name, 0)

        # Normalizar para 0-1000
        score_normalized = max(0, min(1000, int(score)))

        # Intervalo de confianca (bootstrap)
        ci_lower, ci_upper = self._bootstrap_ci(features, n=1000)

        return {
            "score": score_normalized,
            "confidence_interval": [ci_lower, ci_upper],
            "risk_level": self._classify_risk(score_normalized),
            "dimension_breakdown": self._get_breakdown(features),
        }

    def _classify_risk(self, score: int) → str:
        if score ≥ 800: return "BAIXO"
        if score ≥ 600: return "MODERADO"
        if score ≥ 400: return "ALTO"
        return "CRITICO"
```

6.3 API Endpoints

API REST v1.0 - LegalScore PJ

Metodo	Endpoint	Descricao	Timeout
POST	/v1/score/company	Score completo por CNPJ	5s
GET	/v1/company/{cnpj}/profile	Dados cadastrais + CNAE + capital	2s
GET	/v1/company/{cnpj}/processes	Processos com filtros	3s
GET	/v1/company/{cnpj}/risk-breakdown	Detalhamento por dimensao	3s
POST	/v1/batch/score	Bulk scoring (ate 1000 CNPJs)	30s
GET	/v1/audit/{request_id}	Trilha completa do Decision Ledger	2s

6.4 Dockerfile Multi-Stage

```
# docker/products/legalscore/Dockerfile

# === STAGE 1: BUILD ===
FROM python:3.12-slim AS builder
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY services/shared/ /tmp/shared/
COPY services/scoring/requirements.txt /tmp/scoring/
RUN pip install --no-cache-dir --prefix=/install \
    -r /tmp/scoring/requirements.txt

# === STAGE 2: RUNTIME ===
FROM python:3.12-slim AS runtime

RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 curl tini \
    && rm -rf /var/lib/apt/lists/* \
    && groupadd -r appuser && useradd -r -g appuser appuser

WORKDIR /app
COPY --from=builder /install /usr/local
COPY services/shared/ ./shared/
COPY services/scoring/ ./scoring/

HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
    CMD curl -sf http://localhost:8000/health || exit 1

USER appuser
EXPOSE 8000
ENTRYPOINT ["tini", "--"]
CMD ["uvicorn", "scoring.main:app", "--host", "0.0.0.0", \
    "--port", "8000", "--workers", "4"]
```

6.5 Compose Override com Traefik Labels

```
# docker/products/legalscore/compose.override.yml
version: "3.9"

services:
  legalscore-api:
    build:
      context: ../..
      dockerfile: docker/products/legalscore/Dockerfile
    container_name: legalscore-api
    restart: unless-stopped
    environment:
      - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@pgbouncer:6432/legalscore
      - NEO4J_URI=bolt://neo4j:7687
      - NEO4J_USER=neo4j
      - NEO4J_PASSWORD=${NEO4J_PASSWORD}
      - OPENSEARCH_URL=http://opensearch:9200
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/2
      - CHROMA_URL=http://chromadb:8000
      - LLM_API_KEY=${LLM_API_KEY}
      - JWT_SECRET=${JWT_SECRET}
    secrets:
      - jwt_secret
      - db_password
      - llm_api_key
    depends_on:
      pgbouncer:
        condition: service_healthy
      neo4j:
        condition: service_healthy
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.legalscore.rule=Host(`api.legalscore.local`)"
      - "traefik.http.routers.legalscore.entrypoints=websecure"
      - "traefik.http.routers.legalscore.tls.certresolver=letsencrypt"
      - "traefik.http.services.legalscore.loadbalancer.server.port=8000"
    deploy:
      resources:
        limits:
          memory: 4G
          cpus: "2"

secrets:
  jwt_secret:
    file: ./secrets/jwt_secret.txt
  db_password:
    file: ./secrets/db_password.txt
  llm_api_key:
    file: ./secrets/llm_api_key.txt
```

6.6 SLA e Metricas de Producao

SLA e metricas do LegalScore PJ

Metrica	Meta	Como Medir
Disponibilidade	99.5% mensal	Uptime monitoring (Prometheus)
Latencia p95 (API)	< 1.5s	histogram_quantile(0.95, rate(http_request_duration_bucket[5m]))
Latencia p95 (Dashboard)	< 3s	Lighthouse CI + Real User Monitoring
Taxa de erro	< 0.5%	rate(http_requests_total{status=~"5.."}[5m])
Freshness dos dados	< 48h (DATAJUD), < 7d (CAGED)	Timestamp do ultimo ingest por fonte
Throughput sustentado	500 req/s	Locust load test
LLM cache hit ratio	> 60%	redis INFO stats keyspace_hits / (hits + misses)

7. Produto 2: ContabillA v1.0

Sistema de auditoria contabil automatizada. Cruza demonstracoes financeiras com 14 fontes de dados publicos governamentais, identificando inconsistencias com trilha de auditoria completa.

7.1 Arquitetura

O ContabillA recebe demonstracoes financeiras (Balanco Patrimonial, DRE, DLPA) em PDF, Excel ou XBRL, extrai os valores via OCR (Tesseract para PDFs escaneados) ou parsers nativos (openpyxl para Excel, Arelle para XBRL), e entao cruza cada rubrica com dados publicos: numero de empregados declarados vs. vinculos ativos no CAGED, receita declarada vs. execucao orcamentaria no SICONFI, contratos publicos recebidos vs. receita no PNCP, importacoes vs. estoque no ComexStat. Cada inconsistencia gera um registro estruturado com grau de severidade e evidencia documental.

7.2 File Ingest Engine

Parsers suportados pelo File Ingest Engine

Formato	Parser	Tratamento Especial
PDF (digital)	PyMuPDF2 + Camelot (tabelas) + SpaCy (NER)	Extracao de tabelas com coordenadas
PDF (escaneado)	Tesseract OCR + pre-processing	Fontes completas para acentuacao PT-BR
Excel (xlsx/xls)	openpyxl + Pandas	Validacao de schema por tipo de demonstracao
XBRL	Arrelle (SEC + CVM)	Taxonomia CVM para empresas brasileiras
CSV	Pandas com delimiter detection	Encoding detection (chardet)

7.3 Cross-Check Rules

Regras de cross-check implementadas na v1.0

ID	Cross-Check	Fonte Publica	Logica	Severidade Tipica
CC01	Empregados vs. CAGED	CAGED/MTE	Delta > 20% entre folha e vinculos	ALTO
CC02	Receita vs. SICONFI	SICONFI/STN	Delta > 30% (prefeituras)	CRITICO
CC03	Contratos vs. PNCP	PNCP	Receita sem contratos correspondentes	MEDIO
CC04	Importacoes vs. Estoque	ComexStat	Crescimento desproporcional	MEDIO
CC05	Benford's Law (1o digito)	Interno	Chi-quadrado > 15.7 (p < 0.05)	ALTO
CC06	Benford's Law (2o digito)	Interno	Chi-quadrado > 19.2 (p < 0.05)	MEDIO
CC07	Z-score temporal	Historico	Rubrica > 3 sigma da media 12m	ALTO
CC08	Isolation Forest	Multivariado	Score de anomalia > 0.7	CRITICO

7.4 Anomaly Detection

A deteccao de anomalias opera em tres niveis complementares. O primeiro nivel e o Z-score rolling por rubrica, que detecta desvios temporais simples comparando o valor atual com a media e desvio padrao dos ultimos 12 meses. O segundo nivel aplica a Lei de Benford tanto no primeiro quanto no segundo digito significativo de todos os valores monetarios, detectando manipulacao ou fabricacao de dados. O terceiro nivel usa Isolation Forest para deteccao multivariada, identificando combinacoes anormais de rubricas que nao seriam detectadas individualmente. Se o Isolation Forest for lento para o volume de dados, MiniBatchKMeans e a alternativa com fallback automatico.

```

# services/audit/anomaly/detector.py
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.cluster import MiniBatchKMeans

class AnomalyDetector:
    def __init__(self):
        self.iso_forest = IsolationForest(
            n_estimators=100,
            contamination=0.05,
            random_state=42
        )
        self.fallback = MiniBatchKMeans(
            n_clusters=8,
            random_state=42
        )

    def detect(self, features: np.ndarray) → dict:
        try:
            scores = self.iso_forest.decision_function(
                features.reshape(1, -1)
            )[0]
            is_anomaly = self.iso_forest.predict(
                features.reshape(1, -1)
            )[0] == -1
        except Exception:
            # Fallback: K-Means distance
            self.fallback.fit(features.reshape(1, -1))
            scores = -self.fallback.score(features.reshape(1, -1))
            is_anomaly = scores < np.percentile(scores, 5)

        return {
            "anomaly_score": float(scores),
            "is_anomaly": bool(is_anomaly),
            "method": "isolation_forest" if is_anomaly else "kmeans_fallback"
        }

```

8. Produto 3: ComplianceRadar v1.0

Radar de risco contabil-fiscal municipal com alertas proativos multi-canal.

8.1 Arquitetura

O ComplianceRadar monitora continuamente indicadores fiscais e socioeconomicos de municipios configurados pelo cliente. A engine de deteccao de anomalias aplica tres tecnicas complementares: (1) change point detection com Prophet para identificar mudancas estruturais em series temporais, (2) Z-score rolling com ajuste de sazonalidade para detectar desvios de curto prazo, e (3) regressao contra benchmark setorial para contextualizar o desempenho municipal. Os alertas sao classificados em 4 niveis de severidade com deduplicacao temporal (cooldown de 24h para evitar spam).

8.2 Indicadores Monitorados

Indicadores monitorados com logica de alerta

Indicador	Fonte	Formula/Logica	Alerta se...
Arrecadacao municipal	SICONFI	Receita total liquidada	Queda > 20% YoY
Emprego formal	CAGED	Saldo de admissoes - demissoes	Saldo negativo > 3 meses
Indice de Vulnerabilidade (IVM)	Composto (CAGED+ESTBAN)	Media ponderada de 5 sub-indicadores	IVM > percentil 90 nacional
Saneamento basico	SNIS/MDR	Cobertura de agua + esgoto	Cobertura < 50%
Hospitalizacoes respiratorias	DATASUS/SIH	Internacoes CID-10 grupo J por 10k hab	Aumento > 30% YoY
Evasao escolar	INEP + IBGE	1 - (matriculas 7-14 anos / pop 7-14)	Evasao > 10%
Execucao orcamentaria	SICONFI	Liquidado / Empenhado	Taxa < 70% (sub-execucao)
Contratacoes publicas	PNCP	Valor total de contratos por mes	Queda > 40% YoY

8.3 Regras de Alerta Configuraveis

O sistema permite que cada cliente configure regras de alerta customizadas. A sintaxe segue o padrao de expressoes booleanas com operadores aritmeticos. As regras sao avaliadas a cada ciclo de ingest (diario para a maioria dos indicadores), e os alertas gerados passam por deduplicacao temporal (um alerta identico nao e reemitido em 24h) e enriquecimento com contexto jurisprudencial (busca na Central de Inteligencia Juridica por precedentes relevantes ao municipio).

```
# Exemplo de regras de alerta configuradas por cliente
ALERT_RULES = [
  {
    "id": "arrecadacao_critica",
    "name": "Queda critica de arrecadacao",
    "condition": "delta_arrecadacao_yoy < -0.20 AND delta_emprego_yoy < -0.10",
    "severity": "CRITICAL",
    "cooldown_hours": 24,
    "enrichment": True, # Busca jurisprudencia relacionada
    "channels": ["webhook", "email", "slack"],
  },
  {
    "id": "saneamento_baixo",
    "name": "Cobertura de saneamento critica",
    "condition": "cobertura_agua < 50 AND cobertura_esgoto < 30",
    "severity": "HIGH",
    "cooldown_hours": 720, # 30 dias (muda devagar)
    "enrichment": False,
    "channels": ["webhook"],
  },
]
```

8.4 Dashboard Geografico

O dashboard exibe um mapa do Brasil com GeoJSON dos municipios monitorados. Cada municipio e colorido por indicador selecionado (heatmap), com drill-down para detalhes. O mapa usa a malha municipal do IBGE (disponivel no site do IBGE em formato GeoJSON) e e renderizado no frontend com Mapbox GL JS ou Leaflet. O backend fornece os dados via endpoint GeoJSON para integracao com qualquer ferramenta de visualizacao.

9. Produto 4: TaxPredict v1.0

Previsao probabilistica de desfecho de disputas tributarias com modelo bayesiano hierarquico e explicabilidade SHAP.

9.1 Arquitetura

O TaxPredict recebe uma descricao de auto de infracao (texto livre, PDF ou JSON estruturado), classifica a materia tributaria (PIS/COFINS, IRPJ, CSLL, ICMS, IPI, ISS, Simples Nacional), extrai entidades (orgao autuante, periodo, valor, fundamentacao legal), busca acordaos similares no RAG consensual (multi-agentes com weighted voting por especialidade, recencia e vinculancia), enriquece com contexto economico (CAGED por setor, PIB municipal, Selic), e aplica um modelo bayesiano hierarquico para estimar a probabilidade de sucesso com intervalo de credibilidade.

9.2 Taxonomia CARF

Taxonomia de materias tributarias com tribunais e precedentes

Materia	Codigo	Tribunal Principal	Precedente Vinculante
PIS/COFINS	PIS_COFINS	CARF (CSRF)	Solucao de Consulta COSIT
IRPJ	IRPJ	CARF (1a/2a/3a/4a Camara)	Smarmula 340 CTN
CSLL	CSLL	CARF	Smarmula 334 CTN
ICMS	ICMS	STJ (1a Secao)	Obrigacao de pagar x antecipacao
IPI	IPI	CARF	IPI seletivo x ad valorem
ISS	ISS	STJ (1a Secao)	Lista anexa LC 116/2003
Simples Nacional	SIMPLES	CARF + STJ	Sublimite de faturamento

9.3 Modelo Bayesiano Hierarquico

O modelo estima $P(\text{sucesso})$ como funcao de multiplos niveis hierarquicos: (1) nivel nacional (prior baseado em todos os acordaos), (2) nivel do tribunal (bias especifico do CARF/STJ para a materia), (3) nivel da materia (probabilidade base por tipo de tributo), (4) nivel do caso (features especificas como valor, fundamentacao, orgao). A implementacao usa PyMC para amostragem MCMC (NUTS sampler) com 4 chains de 2000 samples cada (1000 warmup + 1000 posterior). O intervalo de credibilidade de 95% e reportado em toda predicao, e valores SHAP indicam quais fatores mais influenciaram o resultado.

```

# services/taxpredict/model/bayesian.py
import pymc as pm
import numpy as np

class TaxPredictionModel:
    def __init__(self, materia: str):
        self.materia = materia
        self.trace = None

    def fit(self, data: pd.DataFrame):
        with pm.Model() as model:
            # Prior hierarquico
            mu_national = pm.Normal('mu_national', mu=0.3, sigma=0.15)
            sigma_tribunal = pm.HalfNormal('sigma_tribunal', sigma=0.1)
            mu_tribunal = pm.Normal('mu_tribunal', mu=mu_national,
                                    sigma=sigma_tribunal,
                                    dims=['tribunal'])

            # Covariates
            beta_valor = pm.Normal('beta_valor', mu=0, sigma=0.1)
            beta_recencia = pm.Normal('beta_recencia', mu=0, sigma=0.1)
            beta_economico = pm.Normal('beta_economico', mu=0, sigma=0.05)

            # Likelihood
            p = pm.math.sigmoid(
                mu_tribunal[data['tribunal_idx']]
                + beta_valor * data['valor_log']
                + beta_recencia * data['recencia']
                + beta_economico * data['indicador_economico']
            )
            obs = pm.Bernoulli('obs', p=p, observed=data['sucesso'])

            self.trace = pm.sample(
                draws=1000, tune=1000, chains=4,
                target_accept=0.9, random_seed=42
            )

    def predict(self, case_features: dict) → dict:
        ppc = pm.sample_posterior_predictive(
            self.trace, var_names=['obs'],
            samples=2000
        )
        mean_prob = ppc['obs'].mean()
        ci_lower = np.percentile(ppc['obs'], 2.5)
        ci_upper = np.percentile(ppc['obs'], 97.5)
        return {
            "probability": round(float(mean_prob), 4),
            "ci_lower": round(float(ci_lower), 4),
            "ci_upper": round(float(ci_upper), 4),
        }

```

10. Produtos Secundarios (Fase 4)

Os quatro produtos secundarios sao essencialmente combinacoes dos engines dos quatro produtos primarios com interfaces especificas. O custo marginal e real (reducao de 60%) porque o stack compartilhado ja existe. Cada produto secundario reutiliza 2-3 engines existentes e adiciona apenas sua logica de negocio e interface.

Produtos secundarios: engines reutilizados e esforco

Produto	Engines Reutilizados	Logica Especifica	Esforco Estimado
LicitaWatch	ComplianceRadar (anomaly) + ContabillA (cross-check) + Ingest PNCP	Monitoramento diario de licitacoes com analise de execucao orcamentaria do orgao	4-5 semanas
DanoBot	ComplianceRadar (IVM + indicadores) + TaxPredict (RAG) + ContabillA (report gen)	Laudo social automatizado a partir de dados socioeconômicos oficiais do municipio	3-4 semanas
PetiBot	TaxPredict (RAG consensual) + DanoBot (context) + AI Engine (geracao)	Minuta de peca processual com citacoes jurisprudenciais + dados empiricos municipais	4-5 semanas
ConciliaIA	LegalScore (scoring) + ComplianceRadar (indicadores) + TaxPredict (predicao)	Faixas de valor para acordo fundamentadas em dados economicos reais do municipio	3-4 semanas

10.1 LicitaWatch - Detalhamento

O LicitaWatch monitora o PNCP diariamente, identifica licitacoes relevantes com base no perfil de atuacao do cliente (CNAEs de interesse, faixa de valor, uf), e realiza analise previa: (1) historico de contratacoes do orgao via SICONFI, (2) indicador OndeFoi (liquidado vs. empenhado) para avaliar se o orgao efetivamente executa contratos, (3) dinamica economica do municipio via CAGED + BCB EST-BAN. Os alertas sao entregues via webhook, email, Slack ou WhatsApp conforme configuracao do cliente.

10.2 DanoBot - Detalhamento

O DanoBot compila automaticamente o perfil socioeconomico completo de um municipio usando os indicadores do DadoSabedoria: IVM (vulnerabilidade), AguaViva (saneamento), Sentinela Respiratoria (saude), Radar de Evasao (educacao), Salario Radar (emprego). Com esses dados, gera um laudo automatizado que fundamenta o valor pretendido ou a prova do dano em acoes coletivas. O laudo e gerado em PDF com trilha de evidencia (fonte + data + valor) para cada indicador citado.

10.3 TribunaConnect - Sala de Guerra Multi-Escritorio

O TribunaConnect é o produto mais ambicioso da Fase 4: uma plataforma colaborativa onde múltiplos escritórios compartilham uma "sala de guerra" para causas coletivas. Recursos: (1) sessões compartilhadas com memória persistente, (2) cada membro recebe alertas no seu canal preferido (WhatsApp, Slack, Telegram), (3) o agente monitora proativamente novidades em jurisprudência e dados socioeconômicos relevantes a causa, (4) LearningRouter aprende o perfil da causa ao longo do tempo e melhora a relevância dos alertas.

11. Caminho de Expansao: Docker para Kubernetes

11.1 Matriz de Decisao: Quando Migrar

Gatilhos de migracao para Kubernetes

Sinal	Metrica Limite	Acao
CPU sustentado > 70 %	Qualquer container por > 1 hora	Escalar horizontalmente - momento de K8s
Multiplas instancias necessarias	> 3 instancias do mesmo servico	K8s com HPA (Horizontal Pod Autoscaler)
Zero-downtime deploys	Necessidade de rollback automatico	K8s com rolling updates
Multi-region / multi-AZ	Disponibilidade em > 1 regioao	K8s com cluster federation
Complexidade operacional	> 50 containers no compose	K8s (compose fica impossivel de gerenciar)
Pod security policies	Necessidade de isolamento por pod	K8s com OPA/Gatekeeper

11.2 Estrategia de Migracao por Componente

Evolucao de cada componente atraves das 3 fases

Fase Atual (Docker)	Fase 2 (K8s Nativo)	Fase 3 (Managed Service)
PostgreSQL (container)	PostgreSQL (StatefulSet)	RDS / Cloud SQL
Neo4j Enterprise (container)	Neo4j (Helm chart)	AuraDB / Neo4j Cloud
OpenSearch (container)	OpenSearch (Helm)	OpenSearch Service
Redis + Sentinel (container)	Redis (Helm + Sentinel)	ElastiCache / Redis Cloud
MinIO (container)	MinIO Operator	S3 / GCS
ChromaDB (container)	Qdrant (Helm)	Pinecone / Zilliz Cloud
Traefik (container)	NGINX Ingress Controller	API Gateway (AWS/GCP)
Celery Beat (container)	Airflow (Helm)	MWAA / Cloud Composer

11.3 Preparacao para K8s Hoje (Sem Custos Agora)

Todas as decisoes de design tomadas hoje no Docker Compose facilitam a migracao futura para Kubernetes sem mudanca de codigo. Os sete principios de preparacao sao: (1) toda configuracao via environment variables, nunca hardcoded, (2) health checks em todo container com endpoint `/health` que verifica dependencias, (3) configs via `.env` hoje viram ConfigMaps e K8s Secrets amanha, (4) persistent volumes nomeados no compose viram PVCs no K8s, (5) multi-stage builds com imagens finais menores que 200MB, (6) resource limits em todo container (ja traduz direto para requests/limits), (7) labels do Traefik viram Ingress resources com anotacoes.

```
# infra/helm/legalscore/values.yaml
# Esqueleto preparatorio para migracao K8s

replicaCount: 1
image:
  repository: ghcr.io/org/legalscore-api
  tag: "1.0.0"
  pullPolicy: IfNotPresent

resources:
  requests:
    cpu: "1"
    memory: 2Gi
  limits:
    cpu: "2"
    memory: 4Gi

autoscaling:
  enabled: false # Habilitar quando CPU > 70%
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: api.legalscore.io
      paths:
        - path: /
  tls:
    - secretName: legalscore-tls
      hosts:
        - api.legalscore.io

# Na migracao: habilitar externalDatabase para RDS
postgresql:
  enabled: true
  auth:
    database: legalscore
```

[i] Compose to K8s Translation

A migracao de Docker Compose para K8s e essencialmente mecanica: `docker-compose.yml` vira `Helm values.yaml`, `Dockerfile` permanece identico, `.env` vira `kubectrl create secret generic`, e `docker volume` vira `PersistentVolumeClaim`. O codigo da aplicacao nao muda.

12. Monitoramento e Observabilidade

12.1 Data Pipeline Health

Painel de saude do pipeline de dados

Metrica Prometheus	Alerta se...	Acao Automatica
ingest_records_per_hour	< 50 % da media dos ultimos 7 dias	Notificar ops + verificar API fonte
data_source_latency_seconds	> 60s para qualquer fonte	Fallback para cache + retry agendado
failed_ingest_tasks_total	> 10 falhas/hora	Escalar worker + alerta Slack
llm_cache_hit_ratio	< 60 %	Revisar策略 de cache + TTL
celery_queue_length	> 1000 tasks pendentes	Escalar workers horizontalmente

12.2 AI Engine Performance

Painel de performance do motor de IA

Metrica	Alerta se...	Acao
llm_response_time_seconds (p95)	> 15s	Mudar para modelo mais rapido + cache
llm_cost_per_day	> orcamento configurado	Throttle de requests + alerta financeiro
rag_faithfulness_score	< 0.7	Revisar prompt template + fontes
consensus_disagreement_rate	> 30 %	Investigar divergencia entre agentes
embedding_latency_seconds	> 2s	Verificar ChromaDB health + CPU

12.3 Business KPIs

KPIs de negocio monitorados

Metrica	Descricao	Fonte
score_requests_per_day	Volume de uso da API	API Gateway logs
top_cnae_by_requests	Setores mais consultados	API request params
avg_risk_score	Tendencia macro de risco	PostgreSQL aggregates
municipality_alerts_per_day	Volume de alertas ComplianceRadar	Alert engine logs
audit_reports_generated	Relatorios ContabillA gerados	Report generator logs
prediction_accuracy	Acuracia do TaxPredict vs. resultado real	Feedback loop

12.4 Stack de Monitoramento (Docker Compose)

```
# docker/compose/monitoring.yml

prometheus:
  image: prom/prometheus:v2.50
  container_name: prometheus
  restart: unless-stopped
  volumes:
    - ../config/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - "--config.file=/etc/prometheus/prometheus.yml"
    - "--storage.tsdb.retention.time=30d"
  ports:
    - "9090:9090"

grafana:
  image: grafana/grafana:10.3
  container_name: grafana
  restart: unless-stopped
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
    - GF_USERS_ALLOW_SIGN_UP=false
  volumes:
    - grafana_data:/var/lib/grafana
    - ../config/grafana/provisioning:/etc/grafana/provisioning
  ports:
    - "3001:3000"

loki:
  image: grafana/loki:2.9
  container_name: loki
  restart: unless-stopped
  volumes:
    - loki_data:/loki
  ports:
    - "3100:3100"

promtail:
  image: grafana/promtail:2.9
  container_name: promtail
  restart: unless-stopped
  volumes:
    - /var/log:/var/log:ro
    - /var/lib/docker/containers:/var/lib/docker/containers:ro
    - ../config/promtail.yml:/etc/promtail/config.yml
  command: -config.file=/etc/promtail/config.yml
  depends_on:
    - loki

alertmanager:
  image: prom/alertmanager:v0.27
  container_name: alertmanager
  restart: unless-stopped
  volumes:
    - ../config/alertmanager.yml:/etc/alertmanager/alertmanager.yml
  ports:
    - "9093:9093"
```

13. Roadmap Refinado com Gatilhos

13.1 Fase 0 - Setup e Bootstrap (Semanas 1-3)

Fase 0 - Entregas e verificacoes

Sem.	Entrega	Verificacao	Gatilho de Revisao
1	Compose base completa (13 servicos)	make up levanta tudo, make health = healthy	Se algum container OOM: ajustar limits
1	Docker Secrets para producao	secrets/ com permissoes 600	Se dev: .env com direnv
2	Pipeline de ingest minimo (DATAJUD + CAGED)	Celery workers processando dados reais	Se DATAJUD falhar: implementar fallback cache
2	Health checks em TODOS os containers	make health retorna healthy para todos	Se health check falhar: debug dependencias
3	API Gateway + JWT + rate limiting	curl localhost:8000/docs mostra Swagger	Se rate limit bloquear legitimo: ajustar
3	Primeiro load test (50 req/s)	Locust sustem 50 req/s por 2 min	Se p95 > 2s: investigar bottleneck

13.2 Fase 1 - LegalScore PJ v1.0 (Semanas 4-13)

Fase 1 - Entregas e gatilhos de revisao

Sem.	Entrega	Gatilho de Revisao
4-5	Neo4j schema + seed (5M+ nos)	Se ingest inicial > 24h: paralelizar com multiprocessing
5-6	Score Engine MLR (100 modelos CNAE)	Se dados de calibracao insuficientes: Delphi method
6-7	Cross-check com CAGED/SICONFI/PGFN	Se API publica sem dados granulares: agregado + estimativa
7-8	Decision Ledger com Merkle tree	Se volume > 1M registros/mes: PG partitioned tables
8-10	Dashboard Next.js + TanStack Query	Se > 500 downloads/mes: async export com Celery
10-12	API v1.0 (6 endpoints OpenAPI 3.1)	Se p95 > 1.5s: Redis cache no score engine
12-13	QA + Load test (500 req/s) + Pentest	Se OWASP ZAP encontrar criticos: corrigir antes do piloto

13.3 Fase 2 - ContabillA v1.0 (Semanas 14-23)

Fase 2 - Entregas e pontos criticos

Sem.	Entrega	Ponto Critico
14-15	OCR pipeline (Tesseract + validation)	Tesseract precisa de fontes completas para PDFs escaneados com acentuacao
16-18	Cross-check engine (6 fontes)	Se CAGED nao tem CNPJ granular: usar CNAE agregado + estimativa proporcional
19-21	Anomaly detection (Z-score + Benford + IF)	Se Isolation Forest lento: fallback para MiniBatchKMeans
21-23	Report generator + HITL review flow	Implementar safety rail: auditor revisa antes de exportar

13.4 Fase 3 - ComplianceRadar + TaxPredict (Semanas 24-39)

Fase 3 - Milestones por produto

Sem.	Produto	Milestone
24-27	ComplianceRadar	IVM calculado para 5.570 municipios com cadencia mensal
28-31	ComplianceRadar	Alertas com SLA de 99% (fallback para cache se fonte fora)
32-35	TaxPredict	Bayesian model calibrado com >= 2.000 acordaos por materia
36-39	TaxPredict	SHAP values + credible intervals em toda previsao + dashboard de calibracao

13.5 Fase 4 - Produtos Secundarios + K8s (Semanas 40-52)

Fase 4 - Produtos secundarios e migracao K8s

Sem.	Produto	Milestone
40-44	LicitaWatch + DanoBot	Dois produtos reutilizando engines existentes
44-48	PetiBot + ConciliaIA	Dois produtos reutilizando engines existentes
48-50	TribunaConnect	Plataforma colaborativa multi-escritorio
50-52	K8s Migration	Helm charts + StatefulSets + Ingress + HPA

14. Hardware, Custos e Riscos

14.1 Servidor Unico - Fase 0-2

Ate o final da Fase 2 (semana 23), toda a plataforma roda em um unico servidor com as especificacoes abaixo. Este servidor acomoda todos os 13 servicos do stack compartilhado, os containers dos produtos 1 e 2, os workers de ingest, e o frontend de desenvolvimento. A estimativa de custo considera provedores como Hetzner, DigitalOcean ou AWS EC2.

Especificacoes do servidor unico (Fase 0-2)

Recurso	Especificacao	Justificativa
CPU	16 vCPU	Neo4j (4) + OpenSearch (2) + API (2) + Celery (4) + PG (2) + overhead
RAM	64 GB DDR4/5	Neo4j heap (4G) + PG (4G) + OpenSearch (4G) + Redis (2G) + API (4G) + Celery (8G) + Ollama (8G) + OS (8G)
SSD	1 TB NVMe	Neo4j (200G) + PG (100G) + OpenSearch (200G) + MinIO (200G) + logs (100G)
Rede	100 Mbps+	Ingest diario DATAJUD (~2GB/dia) + API traffic
Custo Estimado	R\$ 2.000-4.000/mes	Hetzner (mais barato) / DigitalOcean / AWS EC2 i3.xlarge

14.2 Cluster - Fase 3-4

Cluster de 3 nos (Fase 3-4)

Servidor	Papel	Specs	Custo Estimado
node-1	Databases + Cache	16 vCPU, 64 GB RAM, 1 TB NVMe	R\$ 2.000-4.000/mes
node-2	Application + Workers	16 vCPU, 64 GB RAM, 500 GB SSD	R\$ 2.000-3.500/mes
node-3	Ingest + ML + Ollama	8 vCPU, 32 GB RAM, 500 GB SSD	R\$ 1.000-2.000/mes
TOTAL	3 nos com redundancia	40 vCPU, 160 GB RAM, 2 TB	R\$ 5.000-9.500/mes

14.3 Riscos e Mitigacoes

Riscos identificados com probabilidade, impacto e mitigacao

Risco	Prob.	Impacto	Mitigacao
DATAJUD API instavel	Alta	Alto	Retry exponencial + cache 48h + score parcial com flag
CAGED dados agregados apenas	Media	Medio	CAGED microdados via FTP do MTE; agregado + estimativa proporcional
Neo4j Community nao escala	Media	Alto	Archive de entidades inativas para Parquet/MinIO; migrar para AuraDB
Custo LLM por consulta	Media	Medio	Cache embeddings + modelo local para classificacao + prompt otimizado
LGPD - dados pessoais no grafo	Alta	Alto	Pseudonimizar no ingest; hash; right-to-erasure com soft + hard delete
Calibracao MLR sem labels	Alta	Alto	Delphi method com especialistas; dados de falencia/recuperacao publicos
Tesseract OCR impreciso	Media	Medio	Fontes PT-BR completas; fallback para input manual do auditor
Modelo bayesiano overfitting	Media	Medio	Regularizacao hierarquica; validacao cruzada; Brier score monitoring

15. Checklist de Producao e Validacao

15.1 Checklist Pre-Piloto (Obrigatorio)

Checklist obrigatorio antes de qualquer deploy para cliente piloto

#	Item	Verificacao	Fase
1	TLS real em producao	Let's Encrypt ou certificado comercial; NUNCA HTTP	0
2	Backup automatizado testado	PostgreSQL + Neo4j + MinIO; restore testado semanalmente	0
3	Rate limiting por tenant	100 req/min por tenant; nao derrubar infra com 1 cliente	0
4	CORS policias restritivas	Origins especificos, nunca *	0
5	WAF basico	Cloudflare ou AWS WAF com regras OWASP Top 10	1
6	Logging estruturado JSON	Todos os servicos em JSON > Loki; sem texto plano	0
7	Tracing distribuido	OpenTelemetry em todos os servicos	1
8	Documentacao OpenAPI 3.1	5 exemplos reais por endpoint + schemas	1
9	Onboarding self-service	Tutorial interativo no dashboard	1
10	Playbook de incidentes	Procedimentos para DATAJUD fora, Neo4j OOM, etc.	1
11	Pentest basico (OWASP ZAP)	Zero vulnerabilidades criticas/altas	1
12	Load test 500 req/s sustentado	Locust: 5 min a 500 req/s, p95 < 1.5s	1
13	LGPD: right-to-erasure	Delecao completa em < 30 dias apos solicitacao	1
14	Decision Ledger integridade	Merkle tree valida apos 10k entries	1

15.2 Validacao: Primeiras 72 Horas

Dia 1 - Setup e Bootstrap

- `git clone [repo] && cd juridico-platform`
- Criar secrets: `mkdir secrets && echo 'valor' > secrets/jwt_secret.txt && chmod 600 secrets/*`
- Levantar infraestrutura: `make up`
- Verificar health checks: `make health` - deve retornar healthy para todos
- Rodar migracoes: `make migrate`
- Iniciar ingest inicial DATAJUD (ultimos 30 dias): `make ingest-datajud`

Dia 2 - Primeira Consulta Real

1. Verificar se o ingest rodou: `docker compose logs celery-worker | grep 'ingest completed'`
2. Consultar CNPJ real via API: `POST /v1/score/company {"cnpj": "..."}'`
3. Verificar Decision Ledger: query no PostgreSQL pelos ultimos registros
4. Verificar grafo Neo4j: `MATCH (c:CNPJ {cnpj: '...'}) RETURN c;`
5. Testar cross-check: consultar perfil com dados CAGED+SICONFI

Dia 3 - Load Testing e Otimizacao

1. Load test: `make load-test` (500 req/s por 5 min)
2. Verificar Grafana: `http://localhost:3001` - metricas de latencia, throughput, erros
3. Verificar cache hit ratio: `docker compose logs | grep 'LLM cache'`
4. Verificar Prometheus: histogramas de latencia p50/p95/p99
5. Ajustar resource limits se necessario com base nos resultados

[OK] Sucesso nas 72 Horas

Se ao final das 72 horas todos os itens acima passarem, a plataforma esta pronta para onboarding do primeiro cliente piloto. Qualquer falha deve ser documentada e resolvida antes de prosseguir.

15.3 Makefile de Operacao Diaria

```
# Makefile - Operacao diaria da plataforma
.PHONY: help up down logs migrate test load-test backup health

help:      ## Mostra este help
up:        ## Levanta toda a plataforma (make up)
down:      ## Para toda a plataforma
logs:      ## Logs de todos os servicos
migrate:   ## Roda migrações do banco (Alembic)
test:      ## Roda todos os testes
load-test: ## Teste de carga com Locust (500 req/s, 5 min)
backup:    ## Backup completo (PG + Neo4j + MinIO)
health:    ## Health check de todos os servicos
ingest-datajud: ## Dispara ingest manual do DATAJUD
ingest-caged:  ## Dispara ingest manual do CAGED
ingest-all:   ## Dispara todos os ingests
dash:        ## Abre Grafana + Prometheus + Flower
```